

第五讲 预处理方法

1. 预处理 Krylov 子空间方法
2. 基于矩阵分裂的预处理技术
3. 基于不完全分解的预处理子
4. 基于稀疏近似逆的预处理子
5. 其他预处理技术

为什么预处理

Nothing will be **more central to computational science** in the next century than the art of transforming a problem that appears intractable into another whose solution can be approximated rapidly. For Krylov subspace matrix iterations, this is **preconditioning**.

— Trefethen & Bau, 1997.

- 为什么预处理：改善迭代法的**收敛性与可靠性**.
- 预处理的根本目的：将**难求解**的原问题转化成等价的**容易求解**的新问题.
- 1968 年, Evans 首先提出 “**preconditioning**”, 并考虑了不完全 LU 分解预处理.
- 预处理技术的研究是目前科学计算领域中的重要研究课题之一.

1 预处理 Krylov 子空间方法

1.1 怎么做预处理

1.2 预处理 CG 方法

1.3 预处理 GMRES 方法

1.4 左、右预处理方式的最优性质

1.5 Flexible GMRES 方法

1.6 预处理 BiCGSTAB 方法

1.1

怎么做预处理

预处理方法

对于线性方程组而言, 预处理就是通过适当的线性变换, 将原系数矩阵转换为一个新的矩阵.

考虑线性方程组

$$Ax = b, \quad A \in \mathbb{R}^{n \times n} \text{ 非奇异}, \quad b \in \mathbb{R}^n. \quad (5.1)$$

在方程组的两边同时左乘一个非奇异矩阵 P 的逆, 则可得

$$P^{-1}Ax = P^{-1}b \quad (5.2)$$

这个新方程组就是预处理后的方程组, P 就称为**预处理子** (preconditioner).

当 Krylov 子空间方法被用于求解 (5.2) 时, 就称为**预处理 Krylov 子空间方法**.

好的预处理子

理论上讲, 任何一个非奇异矩阵都可以作为预处理子.

但一个好的预处理子 P 通常需满足下面两个要求:

(1) P 容易构造且 $Pz = r$ 容易求解 $\rightarrow$ 实用

(2) 大大提高迭代法的收敛速度 $\rightarrow$ 有效

($P^{-1}A$ 具有更小的条件数, 更好的特征值分布, 次数更低的最小多项式)

三种不同的预处理方式

🔊 预处理方式 (5.2) 称为 **左预处理**, 即从左边做预处理.

🔊 相应地, 可以对系数矩阵右乘 P^{-1} , 这就是 **右预处理**, 即

$$AP^{-1}u = b, \quad x = P^{-1}u \quad (5.3)$$

🔊 另外, 如果 P 可以分解成两个矩阵的乘积, 或者 P 是由两个矩阵的乘积构成, 即 $P = P_L P_R$, 则可以用下面的方式对原方程组进行预处理

$$P_L^{-1}AP_R^{-1}u = L^{-1}b, \quad x = P_R^{-1}u \quad (5.4)$$

这就是 **两边预处理**

三种预处理方式的几点说明

以上三种方式预处理后的系数矩阵分别为 $P^{-1}A$, AP^{-1} 和 $P_L^{-1}AP_R^{-1}$:

- 具有相同的特征值分布 (三个矩阵是相似的)
- 如果 A 和 P 都是对称正定的, 则使用共轭梯度法求解时, 这三种方式的预处理效果基本上是一样的.
- 对于非对称 (特别是非正规) 情形, 效果可能会相差很大.

三种预处理方式的几点说明（续）

选取哪种预处理方式, 需要根据问题本身和所用的方法来确定:

- 对称正定线性方程组的 CG 方法, 三种方式都可以;
- 非对称的 Krylov 子空间方法, 如 GMRES, 则右预处理比较合适: 一方面是得到的残量范数与原方程组的残量范数是一样的; 另一方面, 极小化的是原始残量范数, 而不是预处理后的残量.

1.2

预处理 CG 方法

设 A 对称正定, 为了保证预处理后的系数矩阵仍然是对称正定的, 通常要求预处理子 P 也是对称正定的.

考虑使用两边预处理方式. 设 P 的 Cholesky 分解为

$$P = LL^T.$$

于是我们得到下面的预处理方程组

$$L^{-1}A(L^T)^{-1}u = L^{-1}b, \quad x = (L^T)^{-1}u. \quad (5.5)$$

近似解记为 $u^{(k)}$, **预处理残量** 记为 $\tilde{r}_k \triangleq L^{-1}b - L^{-1}A(L^\top)^{-1}u^{(k)}$.

算法 1.1 两边预处理 CG 方法

- 1: 给定初值 $x^{(0)}$, 计算 $r_0 = b - Ax^{(0)}$
 - 2: 令 $\tilde{r}_0 = L^{-1}r_0$, $\tilde{p}_1 = \tilde{r}_0$
 - 3: **for** $k = 1, 2, \dots$ **do**
 - 4: $\xi_k = \frac{(\tilde{r}_{k-1}, \tilde{r}_{k-1})}{(L^{-1}A(L^\top)^{-1}\tilde{p}_k, \tilde{p}_k)}$
 - 5: $u^{(k)} = u^{(k-1)} + \xi_k \tilde{p}_k$
 - 6: $\tilde{r}_k = \tilde{r}_{k-1} - \xi_k L^{-1}A(L^\top)^{-1}\tilde{p}_k$
 - 7: $\mu_k = \frac{(\tilde{r}_k, \tilde{r}_k)}{(\tilde{r}_{k-1}, \tilde{r}_{k-1})}$
 - 8: $\tilde{p}_{k+1} = \tilde{r}_k + \mu_k \tilde{p}_k$
 - 9: **end for**
-

原方程组残量

PCG 得到的是预处理后的近似解 $u^{(k)}$ 和残量 $\tilde{r}_k$, 因此我们还需要考虑原方程组的近似解和残量的计算.

对上述算法中进行适当的改写. 首先引入一个辅助变量 p_k :

$$p_k \triangleq (L^\top)^{-1} \tilde{p}_k, \quad k = 1, 2, \dots$$

于是有

$$p_{k+1} = (L^\top)^{-1} p_{k+1} = (L^\top)^{-1} \tilde{r}_k + \mu_k (L^\top)^{-1} \tilde{p}_k = (L^\top)^{-1} \tilde{r}_k + \mu_k p_k.$$

由 $\tilde{r}_k$ 的表达式可知, 原方程组的残量 $r_k = L\tilde{r}_k$, 所以有

$$\begin{aligned} r_k &= L\tilde{r}_k = L(\tilde{r}_{k-1} - \xi_k L^{-1} A (L^\top)^{-1} \tilde{p}_k) = r_{k-1} - \xi_k A p_k, \\ p_{k+1} &= (L^\top)^{-1} \tilde{r}_k + \mu_k p_k = (L^\top)^{-1} L^{-1} r_k + \mu_k p_k = P^{-1} r_k + \mu_k p_k, \end{aligned}$$

$$x^{(k)} = (L^\top)^{-1}u^{(k)} = (L^\top)^{-1}(u^{(k-1)} + \xi_k \tilde{p}_k) = x^{(k-1)} + \xi_k p_k,$$

其中

$$\begin{aligned}\xi_k &= \frac{(L^{-1}r_{k-1}, L^{-1}r_{k-1})}{(L^{-1}A(L^\top)^{-1}\tilde{p}_k, \tilde{p}_k)} = \frac{(r_{k-1}, (L^\top)^{-1}L^{-1}r_{k-1})}{(A(L^\top)^{-1}\tilde{p}_k, (L^\top)^{-1}\tilde{p}_k)} = \frac{(r_{k-1}, P^{-1}r_{k-1})}{(Ap_k, p_k)}, \\ \mu_k &= \frac{(L^{-1}r_k, L^{-1}r_k)}{(L^{-1}r_{k-1}, L^{-1}r_{k-1})} = \frac{(r_k, P^{-1}r_k)}{(r_{k-1}, P^{-1}r_{k-1})}.\end{aligned}$$

记 $z_k \triangleq P^{-1}r_k$, 则可得下面的预处理 CG 方法.

算法 1.2 预处理 CG 方法 (PCG)

- 1: 给定初值 $x^{(0)}$, (相对) 精度要求 $\varepsilon > 0$ 和最大迭代步数 IterMax
- 2: 计算 $r_0 = b - Ax^{(0)}$ 和 $\beta = \|r_0\|_2$
- 3: 令 $z_0 = P^{-1}r_0$, $p_1 = z_0$
- 4: 计算 $\rho = r_0^\top z_0$
- 5: **for** $k = 1$ to IterMax **do**
- 6: $q_k = Ap_k$
- 7: $\xi_k = \rho / (p_k^\top q_k)$
- 8: $x^{(k)} = x^{(k-1)} + \xi_k p_k$
- 9: $r_k = r_{k-1} - \xi_k q_k$
- 10: relres = $\|r_k\|_2 / \beta$
- 11: **if** relres $< \varepsilon$ **then**

```
12:         break
13:     end if
14:      $\rho_0 = \rho$ 
15:      $z_k = P^{-1}r_k$ 
16:      $\rho = r_k^\top z_k$ 
17:      $\mu_k = \rho / \rho_0$ 
18:      $p_{k+1} = z_k + \mu_k p_k$ 
19: end for
20: if relres <  $\varepsilon$  then
21:     输出近似解  $x^{(k)}$  及相关信息
22: else
23:     输出算法失败信息
24: end if
```

注记

我们注意到, 矩阵 L 并没有出现在算法中, 这意味着我们并不需要对 P 做 Cholesky 分解.

注记

在算法 1.2 中, 每步迭代都需要计算 $z_k = P^{-1}r_k$. 一般情况下, 都是通过求解方程组 $Pz_k = r_k$ (广义残量方程) 来实现, 除非 P^{-1} 非常容易得到, 或者是直接给出的.

左预处理方式

事实上, PCG 也可以通过左预处理方式导出. 考虑左预处理方程组

$$P^{-1}Ax = P^{-1}b. \quad (5.6)$$

易知 $P^{-1}A$ 是正定的, 但通常不是对称的, 因此我们不能直接对 (5.6) 实施 CG 方法.

此时我们需要定义一个新的内积: P -内积, 即

$$(x, y)_P \triangleq (Px, y) = (x, Py). \quad (5.7)$$

由于 P 是对称正定的, 所以这个内积定义是有效的.

在该内积下, 有

$$(P^{-1}Ax, y)_P = (Ax, y) = (x, Ay) = (x, P(P^{-1}Ay)) = (x, P^{-1}Ay)_P,$$

即 $P^{-1}A$ 关于 P -内积是自伴随的, 即在 P -内积意义下, $P^{-1}A$ 是“对称”的.

这时可以用 CG 方法来求解预处理方程组 (5.6), 但需要将欧拉内积改为 P -内积. 引入辅助变量 $z_k \triangleq P^{-1}r_k$, 则求解预处理方程组 (5.6) 的 CG 方法可描述如下:

算法 1.3 基于 P -内积的 CG 方法

- 1: 给定初值 $x^{(0)}$, (相对) 精度要求 $\varepsilon > 0$ 和最大迭代步数 IterMax
- 2: 计算 $r_0 = b - Ax^{(0)}$ 和 $\beta = \|r_0\|_2$
- 3: 令 $z_0 = P^{-1}r_0$, $p_1 = z_0$
- 4: **for** $k = 1$ to IterMax **do**
- 5: $\xi_k = \frac{(z_{k-1}, z_{k-1})_P}{(P^{-1}Ap_k, p_k)_P} = \frac{(r_{k-1}, z_{k-1})}{(Ap_k, p_k)}$
- 6: $x^{(k)} = x^{(k-1)} + \xi_k p_k$
- 7: $r_k = r_{k-1} - \xi_k Ap_k$
- 8: $\text{relres} = \|r_k\|_2 / \beta$
- 9: **if** $\text{relres} < \varepsilon$ **then**

```
10:      break
11:  end if
12:   $z_k = z_{k-1} - \alpha_k P^{-1} A p_k = P^{-1} r_k$ 
13:   $\mu_k = \frac{(z_k, z_k)_P}{(z_{k-1}, z_{k-1})_P} = \frac{(r_k, z_k)}{(r_{k-1}, z_{k-1})}$ 
14:   $p_{k+1} = z_k + \mu_k p_k$ 
15: end for
```

注记

不难看出, 算法 1.3 和算法 1.2 是完全一样的.

练习

类似地, 我们也可以从右预处理方式来推导 PCG 方法, 具体过程留作练习.

1.3

预处理 GMRES 方法

左预处理 GMRES

对于非对称情形的 GMRES 方法, 也有三种预处理方式. 但与对称正定情形不同的是, 这三种方式并不等价, 而且有时效果会相差很大.

首先考虑 **左预处理方式**, 即

$$P^{-1}Ax = P^{-1}b \quad (5.8)$$

记 $r_k = b - Ax^{(k)}$ 为原问题的残量, 则预处理方程组 (5.8) 的残量为

$$\tilde{r}_k = P^{-1}r_k$$

将 GMRES 用于求解方程组 (5.8), 即可得 **左预处理 GMRES 方法**.

算法 1.4 左预处理 GMRES 方法

- 1: 给定初值 $x^{(0)}$ 和 (相对) 精度要求 $\varepsilon > 0$
- 2: 计算 $\tilde{r}_0 = P^{-1}(b - Ax^{(0)})$ 和 $\beta = \|\tilde{r}_0\|_2$
- 3: 令 $v_1 = \tilde{r}_0/\beta$, $\xi = \beta e_1$
- 4: **for** $j = 1, 2, \dots$ **do**
- 5: $\tilde{w}_j = Av_j$
- 6: $w_j = P^{-1}\tilde{w}_j$ % Apply preconditioning, solve $Pw_j = \tilde{w}_j$ for w_j
- 7: **for** $i = 1, 2, \dots, j$ **do**
- 8: $h_{ij} = (w_j, v_i)$
- 9: $w_j = w_j - h_{ij}v_i$
- 10: **end for**
- 11: $h_{j+1,j} = \|w_j\|_2$
- 12: **for** $i = 1, 2, \dots, j - 1$ **do** % Apply $G_{j-1}, \dots, G_1$ to last column of $H_{j+1,j}$

```

13:      
$$\begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -s_i & c_i \end{bmatrix} \begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix}$$

14:  end for
15:  if  $h_{j+1,j} = 0$  then
16:      set  $m = j$  and break
17:  end if
18:   $v_{j+1} = w_j / h_{j+1,j}$ 
19:  if  $|h_{j,j}| > |h_{j+1,j}|$  then    % Form the Givens rotation  $G_j$ 
20:       $c_j = \frac{1}{\sqrt{1+\tau^2}}, s_j = c_j \tau$  where  $\tau = \frac{h_{j+1,j}}{h_{j,j}}$ 
21:  else
22:       $s_j = \frac{1}{\sqrt{1+\tau^2}}, c_j = s_j \tau$  where  $\tau = \frac{h_{j,j}}{h_{j+1,j}}$ 
23:  end if
24:   $h_{j,j} = c_j h_{j,j} + s_j h_{j+1,j}, h_{j+1,j} = 0$     % Apply  $G_j$  to last column of  $H_{j+1,j}$ 

```



```

25:    $\begin{bmatrix} \xi_j \\ \xi_{j+1} \end{bmatrix} = \begin{bmatrix} c_j & s_j \\ -s_j & c_j \end{bmatrix} \begin{bmatrix} \xi_j \\ 0 \end{bmatrix}$  % Apply  $G_j$  to the right-hand side
26:   if  $|\xi_{j+1}|/\beta < \varepsilon$  then % Check convergence:  $\|\tilde{r}_j\|_2 = |\xi_{j+1}|$ 
27:       set  $m = j$  and break
28:   end if
29: end for
30: 计算  $y^{(m)} = R_m^{-1}\xi^{(m)}$ , 其中  $R_m = H(1:m, 1:m)$ ,  $\xi^{(m)} = \xi(1:m)$ 
31: 计算近似解  $x^{(m)} = x^{(0)} + V_m y^{(m)}$ 

```

需要指出的是, 在左预处理 GMRES 方法中, 停机准则中的 $|\xi_{j+1}|$ 并不是原方程组残量 r_k 的模, 而是预处理残量 $\tilde{r}_k = P^{-1}(b - Ax^{(k)})$ 的模. 如果要计算真实残量, 除了显式计算外, 没有其它更好的办法. 所以, 如果停机准则是基于真实残量的, 则需要额外去计算 r_k , 这就会增加很多额外的运算量.

右预处理 GMRES

如果用 P 做右预处理, 则对应的预处理方程组为

$$AP^{-1}u = b, \quad x = P^{-1}u. \quad (5.9)$$

给定迭代初值 $x^{(0)}$, 则 $u^{(0)} = Px^{(0)}$. 相应的预处理初始残量为

$$\tilde{r}_0 = b - AP^{-1}u^{(0)} = b - Ax^{(0)} = r^{(0)},$$

即预处理残量与原方程组残量是一样的. 因此无需计算 $u^{(0)}$. 设 $u^{(m)} = u^{(0)} + V_m y_m$ 是迭代 m 步后的近似解, 则对应的原方程组近似解 $x^{(m)}$ 为

$$x^{(m)} = P^{-1}u^{(m)} = P^{-1}(u^{(0)} + V_m y_m) = x^{(0)} + P^{-1}V_m y_m,$$

原方程组的残量为

$$r_m = b - Ax^{(m)} = r_0 - AP^{-1}V_my_m = \beta v_1 - V_{m+1}H_{m+1,m}y_m = V_{m+1}(\beta e_1 - H_{m+1,m}y_m).$$

由于 y_m 是最小二乘问题

$$\min_{y \in \mathbb{R}^m} \|\beta e_1 - H_{m+1,m}y\|_2$$

的解, 因此

$$\|r_m\|_2 = \|\beta e_1 - H_{m+1,m}y_m\|_2 = \beta \cdot |q_1(m+1)|.$$

这与不带预处理的 GMRES 方法是一样的. 因此在实际求解过程中我们可以直接得到原方程组残量的模, 而无需计算 $u^{(m)}$ 和 $x^{(m)}$. 这是右预处理方式与左预处理方式的主要区别.

右预处理 GMRES 方法具体描述如下:

算法 1.5 右预处理 GMRES 方法

- 1: 给定初值 $x^{(0)}$ 和 (相对) 精度要求 $\varepsilon > 0$
- 2: 计算 $r_0 = b - Ax^{(0)}$ 和 $\beta = \|r_0\|_2$
- 3: 令 $v_1 = \tilde{r}_0/\beta, \xi = \beta e_1$
- 4: **for** $j = 1, 2, \dots$ **do**
- 5: $\tilde{w}_j = P^{-1}v_j$ **% Apply preconditioning**
- 6: $w_j = A\tilde{w}_j$
- 7: **for** $i = 1, 2, \dots, j$ **do**
- 8: $h_{ij} = (w_j, v_i)$
- 9: $w_j = w_j - h_{ij}v_i$
- 10: **end for**
- 11: $h_{j+1,j} = \|w_j\|_2$

```

12:   for  $i = 1, 2, \dots, j - 1$  do    % Apply  $G_{j-1}, \dots, G_1$  to the last column of  $H_{j+1,j}$ 
13:       
$$\begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -s_i & c_i \end{bmatrix} \begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix}$$

14:   end for
15:   if  $h_{j+1,j} = 0$  then
16:       set  $m = j$  and break
17:   end if
18:    $v_{j+1} = w_j / h_{j+1,j}$ 
19:   if  $|h_{j,j}| > |h_{j+1,j}|$  then    % Form the Givens rotation  $G_j$ 
20:        $c_j = \frac{1}{\sqrt{1+\tau^2}}, s_j = c_j \tau$  where  $\tau = \frac{h_{j+1,j}}{h_{j,j}}$ 
21:   else
22:        $s_j = \frac{1}{\sqrt{1+\tau^2}}, c_j = s_j \tau$  where  $\tau = \frac{h_{j,j}}{h_{j+1,j}}$ 
23:   end if

```

```

24:       $h_{j,j} = c_j h_{j,j} + s_j h_{j+1,j}, h_{j+1,j} = 0$     % Apply  $G_j$  to last column of  $H_{j+1,j}$ 

25:       $\begin{bmatrix} \xi_j \\ \xi_{j+1} \end{bmatrix} = \begin{bmatrix} c_j & s_j \\ -s_j & c_j \end{bmatrix} \begin{bmatrix} \xi_j \\ 0 \end{bmatrix}$     % Apply  $G_j$  to the right-hand side

26:      if  $|\xi_{j+1}|/\beta < \varepsilon$  then    % Check convergence:  $\|r_j\|_2 = |\xi_{j+1}|$ 

27:          set  $m = j$  and break

28:      end if

29: end for

30: 计算  $y^{(m)} = R_m^{-1} \xi^{(m)}$ , 其中  $R_m = H(1:m, 1:m)$ ,  $\xi^{(m)} = \xi(1:m)$ 

31: 计算近似解  $x^{(m)} = x^{(0)} + P^{-1} V_m y^{(m)}$ 

```

两边预处理方式

如果预处理子 P 是由乘积形式给出的, 即

$$P = P_L P_R,$$

则可以构造下面的两边预处理方程组

$$P_L^{-1} A P_R^{-1} u = P_L^{-1} b, \quad x = P_R^{-1} u.$$

将 GMRES 用于求解上述线性方程组, 则可得到两边预处理 GMRES 方法. 具体推导过程与前面类似, 留作练习.

需要指出的是, 对于两边预处理 GMRES 方法, 其预处理后的残量为

$$\tilde{r}_k = P_L^{-1}(b - Ax^{(k)}).$$

注记

由于左预处理方式和两边预处理方式所得到的残量并不是原方程组的残量, 因此它们都可能会导致迭代提前终止或延迟终止., 特别是当预处理子 P 的条件数较大时. 因此, 通常情况下建议采用右预处理方式.

1.4

左、右预处理方式的最优性质

设 $x^{(0)}$ 是迭代初始值, 记 $x_L^{(k)}$ 和 $x_R^{(k)}$ 分别是左预处理 GMRES 方法和右预处理 GMRES 方法迭代 k 步后得到的近似解. 根据 GMRES 方法的最优性质可知, 对于左预处理 GMRES 方法, 有

$$x_L^{(k)} = \underset{x \in x^{(0)} + \mathcal{K}_k(P^{-1}A, P^{-1}r_0)}{\operatorname{argmin}} \|P^{-1}(b - Ax)\|_2 \quad (5.10)$$

而对于右预处理 GMRES, 我们有 $x_R^{(k)} = P^{-1}u^{(k)}$, 其中

$$u^{(k)} = \underset{u \in u^{(0)} + \mathcal{K}_k(AP^{-1}, r_0)}{\operatorname{argmin}} \|b - AP^{-1}u\|_2.$$

通过变量代换 $x = P^{-1}u$, 可得

$$x_R^{(k)} = \underset{x \in x^{(0)} + P^{-1}\mathcal{K}_k(AP^{-1}, r_0)}{\operatorname{argmin}} \|b - Ax\|_2 \quad (5.11)$$

又

$$\begin{aligned} & P^{-1}\mathcal{K}_k(AP^{-1}, r_0) \\ &= P^{-1} \operatorname{span}\{r_0, AP^{-1}r_0, (AP^{-1})^2r_0, \dots, (AP^{-1})^{k-1}r_0\} \\ &= \operatorname{span}\{P^{-1}r_0, P^{-1}AP^{-1}r_0, P^{-1}(AP^{-1})^2r_0, \dots, P^{-1}(AP^{-1})^{k-1}r_0\} \\ &= \operatorname{span}\{P^{-1}r_0, (P^{-1}A)P^{-1}r_0, (P^{-1}A)^2P^{-1}r_0, \dots, (P^{-1}A)^{k-1}P^{-1}r_0\} \\ &= \mathcal{K}_k(P^{-1}A, P^{-1}r_0), \end{aligned}$$

由 (5.10) 和 (5.11) 可知, $x_L^{(k)}$ 和 $x_R^{(k)}$ 属于同一个子空间中. 不同之处在于, $x_L^{(k)}$ 极小化的是预处理后残量的范数, 即 $\|P^{-1}(b - Ax)\|_2$, 而 $x_R^{(k)}$ 极小化的是原方程组残量的范数, 即 $\|b - Ax\|_2$.

定理 1 设 $x_L^{(k)}$ 和 $x_R^{(k)}$ 分别是左预处理 GMRES 方法和右预处理 GMRES 方法迭代 k 步后得到的近似解, 且迭代初始值均为 $x^{(0)}$. 则 $x_L^{(k)}$ 是 $\|P^{-1}(b - Ax)\|_2$ 在 $x^{(0)} + \mathcal{K}_k(P^{-1}A, P^{-1}r_0)$ 的极小值点, 而 $x_R^{(k)}$ 则是 $\|b - Ax\|_2$ 在同一个子空间中的极小值点.

1.5

Flexible GMRES 方法

在前面的讨论中, 我们假定预处理子 P 是固定不变的. 事实上, 我们可以在每步迭代中取不同的预处理子. 这就是 **Flexible GMRES (FGMRES)** 方法的基本思想. 下面介绍右预处理方式的 Flexible GMRES 方法.

我们知道, 在右预处理 GMRES 方法 1.5 中, 近似解可以写成下面的形式:

$$x^{(m)} = x^{(0)} + P^{-1}V_m\tilde{y} = x^{(0)} + \sum_{j=1}^m (P^{-1}v_j)\tilde{y}_j. \quad (5.12)$$

这意味着 $x^{(m)} - x^{(0)}$ 是向量 $P^{-1}v_1, P^{-1}v_2, \dots, P^{-1}v_m$ 的线性组合. 而这些向量可由算法 1.5 中的第 5 步计算得到. 但事实上, 我们无需存储这些向量, 因为它们是由同一个预处理子 P 作用到向量 v_j 's 上所得到的. 因此, 在计算近似解 $x^{(m)}$ 时, 只需将 P 作用到 $v_1, v_2, \dots, v_m$ 的线性组合上即可, 即 $V_m\tilde{y}$ 上.

但如果每次迭代时采用不同的预处理子, 即将算法 1.5 中的第 5 步改为

$$5': \text{ solve } P_j w = v_j \text{ for } w, \text{ that is, } w = P_j^{-1} v_j$$

因此, 我们很自然地想到用下面的方式来获取近似解:

$$x^{(m)} = x^{(0)} + \sum_{j=1}^m (P_j^{-1} v_j) \tilde{y}_j = x^{(0)} + Z_m \tilde{y},$$

其中 $Z_m = [z_1, z_2, \dots, z_m]$. 这 $z_j = P_j^{-1} v_j$, 向量 $\tilde{y}$ 则跟之前一样计算. 需要注意的是, 此时我们需要存储所有的向量 $z_j = P_j^{-1} v_j, j = 1, 2, \dots, m$. 这就是 FGMRES 与右预处理 GMRES 方法的根本区别.

FGMRES 算法描述如下:

算法 1.6 Flexible GMRES (FGMRES)

- 1: 给定初值 $x^{(0)}$ 和 (相对) 精度要求 $\varepsilon > 0$
- 2: 计算 $r_0 = b - Ax^{(0)}$ 和 $\beta = \|r_0\|_2$
- 3: set $\xi = \beta e_1$ and $v_1 = r_0/\beta$
- 4: **for** $j = 1, 2, \dots$ **do**
- 5: solve $P_j z_j = v_j$ for z_j , that is, $z_j = P^{-1}v_j$ % Apply the preconditioner
- 6: $w_j = Az_j$
- 7: **for** $i = 1, 2, \dots, j$ **do** % Arnoldi process
- 8: $h_{ij} = (w_j, v_i)$
- 9: $w_j = w_j - h_{ij}v_i$
- 10: **end for**
- 11: $h_{j+1,j} = \|w_j\|_2$


```

12:  if  $h_{j+1,j} = 0$  then
13:      set  $m = j$  and break
14:  end if
15:   $v_{j+1} = w_j / h_{j+1,j}$ 
16:  for  $i = 1, 2, \dots, j - 1$  do    % Apply  $G_{j-1}, \dots, G_1$  to the last column of  $H_{i+1,j}$ 
17:      
$$\begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -s_i & c_i \end{bmatrix} \begin{bmatrix} h_{i,j} \\ h_{i+1,j} \end{bmatrix}$$

18:  end for
19:  if  $|h_{j,j}| > |h_{j+1,j}|$  then    % Form the Givens rotation  $G_j$ 
20:      set  $c_j = \frac{1}{\sqrt{1+\tau^2}}$ ,  $s_j = c_j \tau$  where  $\tau = \frac{h_{j+1,j}}{h_{j,j}}$ 
21:  else
22:      set  $s_j = \frac{1}{\sqrt{1+\tau^2}}$ ,  $c_j = s_j \tau$  where  $\tau = \frac{h_{j,j}}{h_{j+1,j}}$ 
23:  end if

```

```

24:    $h_{j,j} = c_j h_{j,j} + s_j h_{j+1,j}, h_{j+1,j} = 0$     % Apply  $G_j$  to last column of  $H_{j+1,j}$ 

25:    $\begin{bmatrix} \xi_j \\ \xi_{j+1} \end{bmatrix} = \begin{bmatrix} c_j & s_j \\ -s_j & c_j \end{bmatrix} \begin{bmatrix} \xi_j \\ 0 \end{bmatrix}$     % Apply  $G_j$  to the right-hand side

26:   if  $\|\xi_{j+1}\|_2 < \varepsilon$  then    % Check convergence:  $\|\tilde{r}\|_2 = \xi_{j+1}$ 
27:       set  $m = j$  and break
28:   end if

29: end for

30: compute  $\tilde{y} = R_m^{-1} \xi_m$  with  $R_m = H(1 : m, 1 : m)$  upper-triangular and  $\xi_m = \xi(1 : m)$ 

31: compute the approximate solution  $x^{(m)} = x^{(0)} + Z_m \tilde{y}$ 

```

1.6

预处理 BiCGSTAB 方法

右预处理 BiCGSTAB

与带预处理的 GMRES 方法类似, 为了使得计算过程中得到的残量是真实残量 (而不是预处理后的残量), 我们一般采用右预处理方式, 即

$$AM^{-1}u = b \quad \text{with} \quad u = Mx, \quad (5.13)$$

其中 M 为预处理子.

在 BiCGSTAB 方法中, 主要迭代过程为:

$$\begin{aligned} q_k &= r_{k-1} - \alpha_k A p_k \quad \text{with} \quad \alpha_k = \frac{(r_{k-1}, \tilde{r}_0)}{(A p_k, \tilde{r}_0)}, \\ r_k &= q_k - \omega_k A q_k \quad \text{with} \quad \omega_k = \frac{(q_k, A q_k)}{(A q_k, A q_k)}, \\ x^{(k)} &= x^{(k-1)} + \alpha_k p_k + \omega_k q_k, \end{aligned}$$

$$p_{k+1} = r_k + \beta_k(p_k - \omega_k A p_k) \quad \text{with } \beta_k = \frac{\alpha_k}{\omega_k} \cdot \frac{(r_k, \tilde{r}_0)}{(r_{k-1}, \tilde{r}_0)},$$

其中 $k = 1, 2, \dots, p_1 = r_0$. 记 $\tilde{p}_k = M^{-1}p_k$, $\tilde{q}_k = M^{-1}q_k$, 则针对预处理后的线性方程组 (5.13) 的 BiCGSTAB 迭代过程为:

$$q_k = r_{k-1} - \alpha_k A \tilde{p}_k \quad \text{with } \alpha_k = \frac{(r_{k-1}, \tilde{r}_0)}{(A \tilde{p}_k, \tilde{r}_0)},$$

$$r_k = q_k - \omega_k A \tilde{q}_k \quad \text{with } \omega_k = \frac{(q_k, A \tilde{q}_k)}{(A \tilde{q}_k, A \tilde{q}_k)},$$

$$x^{(k)} = x^{(k-1)} + \alpha_k \tilde{p}_k + \omega_k \tilde{q}_k,$$

$$p_{k+1} = r_k + \beta_k(p_k - \omega_k A \tilde{p}_k) \quad \text{with } \beta_k = \frac{\alpha_k}{\omega_k} \cdot \frac{(r_k, \tilde{r}_0)}{(r_{k-1}, \tilde{r}_0)}.$$

整理后即可得下面的预处理 BiCGSTAB 方法:

算法 1.7 Preconditioned BiCGSTAB

- 1: Given an initial guess $x^{(0)}$ and stopping tolerance $\varepsilon > 0$
- 2: compute $r_0 = b - Ax^{(0)}$
- 3: choose $\tilde{r}_0$ such that $(r_0, \tilde{r}_0) \neq 0$ % one common choice is $\tilde{r}_0 = r_0$
- 4: set $p_1 = r_0$
- 5: **for** $k = 1, 2, \dots$ **do**
- 6: $\rho_{k-1} = (r_{k-1}, \tilde{r}_0)$ % if $\rho_{k-1} = 0$, then breakdown
- 7: **if** $k > 1$ **then**
- 8: $\beta_{k-1} = (\alpha_{k-1}/\omega_{k-1}) \cdot (\rho_{k-1}/\rho_{k-2})$
- 9: $p_k = r_{k-1} + \beta_{k-1}(p_{k-1} - \omega_{k-1}v_{k-1})$
- 10: **end if**
- 11: $\tilde{p}_k = M^{-1}p_k$ % solve $M\tilde{p}_k = p_k$ for $\tilde{p}_k$

```

12:     $v_k = A\tilde{p}_k$ 
13:     $\alpha_k = \rho_{k-1} / (v_k, \tilde{r}_0)$     % if  $(v_k, \tilde{r}_0) = 0$ , then breakdown
14:     $q_k = r_{k-1} - \alpha_k v_k$ 
15:     $x^{(k)} = x^{(k-1)} + \alpha_k \tilde{p}_k$     % if  $\|q_k\| < \varepsilon$ , then converged and stop
16:     $\tilde{q}_k = M^{-1}q_k$     % solve  $M\tilde{q}_k = q_k$  for  $\tilde{q}_k$ 
17:     $w_k = A\tilde{q}_k$ 
18:     $\omega_k = (q_k, w_k) / (w_k, w_k)$     % if  $\omega_k = 0$ , then breakdown
19:     $r_k = q_k - \omega_k w_k$ 
20:     $x^{(k)} = x^{(k)} + \omega_k \tilde{q}_k$ 
21:    if  $\|r_k\|_2 < \varepsilon$  then    % check convergence
22:        stop    % converged
23:    end if
24: end for

```

2 基于矩阵分裂的预处理技术

Finding a good preconditioner to solve a given sparse linear system is often viewed as a combination of art and science. Theoretical results are rare and some methods work surprisingly well, often despite expectations.

— Saad, 2003.

预处理能否取得成功的关键是能否找到一个好的预处理子. 预处理子的构造与问题本身是密切相关的, 通用的“好预处理子”是不存在的.

关于预处理技术的理论分析很少, 大多数情况下只能根据经验来构造. 尽管如此, 在实际应用中, 这些根据经验构造出来的预处理子往往能取得很好的效果, 有时效果之好会大大出乎人们的意料之外.

一般来说, 预处理子可以分为两大类:

- (a) 代数预处理子 (Algebraic Preconditioner), 即仅仅根据所给方程组的系数矩阵来构造预处理子.
- (b) 专用预处理子 (Problem-Specific Preconditioner), 即根据问题的物理背景所构造的专用预处理子.

显然, 由于专用预处理子充分利用了问题的物理背景知识, 所以它们往往具有很好的数值表现, 如多重网格, 区域分解, 快速变换等等. 但它们严重依赖于原问题的物理背景, 因此不具有通用性.

我们这里只介绍代数预处理子, 即仅仅根据所给的系数矩阵来构造预处理方法. 这种预处理方法具有一定的通用性.

考虑线性方程组

$$Ax = b, \quad A \in \mathbb{R}^{n \times n},$$

对 A 做如下的矩阵分裂:

$$A = M - N \tag{5.14}$$

其中 M 非奇异, 则可以得到下面的迭代方法

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b = x^{(k)} + (M^{-1}b - M^{-1}A)x^{(k)}. \quad k = 0, 1, 2, \dots$$

这等价于求解下面的方程组

$$M^{-1}Ax = M^{-1}b. \tag{5.15}$$

这就是与矩阵分裂 (5.14) 相对应的预处理线性方程组.

将 Krylov 子空间方法用于求解方程组 (5.15), 就得到预处理 Krylov 子空间方法. 矩阵 M 就是由矩阵分裂 (5.14) 所定义的预处理器.

理论上讲, 任何一个矩阵分裂都可以定义一个预处理器. 但为了使得预处理器能有很好的预处理效果, 往往需要其在一定意义下与 A 充分接近.

Jacobi, Gauss-Seidel, SOR, SSOR 所对应的预处理器为

$$P_J = D,$$

$$P_{GS} = D - L,$$

$$P_{SOR} = D - \omega L$$

$$P_{SSOR} = (D - \omega L)D^{-1}(D - \omega U),$$

$$P_{SGS} = (D - L)D^{-1}(D - U) = \tilde{L}\tilde{U},$$